

**Projet : Mise en
place d'un système
de Monitoring et
d'Automatisation
DevOps sous Debian**

Table des matières

1. Présentation du projet	3
2. Architecture du projet.....	3
3. Fichier de configuration	5
4. Gestion des alertes	6
5. Déploiement automatisé	7
6. Monitoring système	7
7. Vérification des services.....	10
8. Nettoyage automatique.....	11
9. Automatisation avec Cron	11
10. Résultat global	12
Conclusion.....	13

1. Présentation du projet

Dans le cadre de ce Projet réalisé sous Debian, l'objectif était de développer un mini système DevOps permettant d'automatiser la supervision et l'administration d'un serveur Linux.

Le système mis en place permet notamment :

- La surveillance des ressources système (CPU, RAM, espace disque)
 - La vérification d'un service critique (Nginx)
 - La génération automatique de rapports
 - L'émission d'alertes en cas d'incident
 - Le redémarrage automatique d'un service défaillant
 - Le déploiement automatisé d'une application
 - Le nettoyage des anciens rapports
 - L'exécution automatique grâce à Cron
-

2. Architecture du projet

Le projet est organisé de manière professionnelle afin de séparer les différentes fonctionnalités.

monitoring-devops/

|

├─ monitor.sh

├─ deploy.sh

├─ alert.sh

├─ config.conf

```
|
|
|└─ utils/
|  └─ functions.sh
|
|
|└─ app/
|  └─ index.html
|
|
|└─ reports/
|
|
└─ logs/
```

Capture d'écran 1 : Structure du projet

```
b04@b04-debian-server:~/Workspace/Reborn-Project/Automatisation/TP3$ tree monitoring-devops/
monitoring-devops/
├── alert.sh
├── app
│   └── index.html
├── config.conf
├── deploy.sh
├── LOG_DIR
├── logs
│   ├── deploy.log
│   └── monitor.log
├── monitor.sh
├── reports
│   ├── report_25-05-2026_15-53_18.txt
│   ├── report_25-05-2026_15-55_45.txt
│   ├── report_25-05-2026_18-41_12.txt
│   ├── report_25-05-2026_18-46_46.txt
│   ├── report_25-05-2026_19-33_56.txt
│   ├── report_25-05-2026_19-42_37.txt
│   ├── report_25-05-2026_19-55_29.txt
│   ├── report_25-05-2026_20-51_57.txt
│   ├── report_25-05-2026_20-56-00.txt
│   └── report_25-05-2026_20-59-01.txt
└── utils
    └── functions.sh

6 directories, 18 files
```

3. Fichier de configuration

Le fichier config.conf centralise l'ensemble des paramètres utilisés par les scripts :

- Service à surveiller
- Seuils CPU, RAM et disque
- Emplacement des rapports
- Emplacement des journaux (logs)

Exemple :

```
SERVICE="nginx"
```

```
CPU_MAX=80
```

```
RAM_MAX=80
```

```
DISK_MAX=85
```

Intérêt

Cette approche permet de modifier facilement la configuration sans toucher au code des scripts.

Capture d'écran 2 : Contenu du fichier de configuration

```
GNU nano 8.4 config.cor
# =====
# CONFIGURATION GLOBALE
# =====

SERVICES=("nginx" "ssh" "ufw" "fail2ban")

CPU_MAX=80
RAM_MAX=80
DISK_MAX=85

RETENTION_DAYS=7

BASE_DIR="$HOME/Workspace/Reborn-Project/Automatisation/TP3/monitoring-devops"
REPORT_DIR="$BASE_DIR/reports"
LOG_DIR="$BASE_DIR/logs"

LOG_FILE="$LOG_DIR/monitor.log"
ALERT_FILE="$LOG_DIR/alert.log"
DEPLOY_LOG="$LOG_DIR/deploy.log"
```

4. Gestion des alertes

Le script alert.sh est responsable de la gestion des alertes.

Lorsqu'un problème est détecté :

- un message est affiché à l'écran ;
- l'évènement est enregistré dans un fichier de log.

Exemple d'alerte :

ALERT: Service nginx arrêté

Capture d'écran 3 : Script du fichier des alertes

```
GNU nano 8.4 alert.sh
#!/bin/bash

source config.conf

send_alert() {
    MESSAGE="$1"
    echo "[$(date '+%d-%m-%Y %H-%M-%S')] ALERT : $MESSAGE" | tee -a "$ALERT_FILE"
}
```

5. Déploiement automatisé

Le script `deploy.sh` simule un processus de déploiement DevOps.

Son rôle est :

- de cloner une application depuis un dépôt Git ;
- ou de mettre à jour le code existant ;
- d'enregistrer les opérations dans un journal.

Commande utilisée :

`./deploy.sh`

6. Monitoring système

Le cœur du projet est le script `monitor.sh`.

Capture d'écran 4 : Script du fichier de monitoring système

```
GNU nano 8.4 monitor.sh
#!/bin/bash

source config.conf
source utils/functions.sh
source alert.sh

mkdir -p "$REPORT_DIR" "LOG_DIR"

DATE=$(date +"d-%m-%Y_%H-%M-%S")
REPORT_FILE="$REPORT_DIR/report_$DATE.txt"

# =====
# COLLECTE METRIQUES
# =====

CPU_USAGE=$(LC_ALL=C top -bn1 | grep "Cpu(s)" | awk '{print $2 + $4}')
RAM_USAGE=$(free | awk '/Mem:/ {printf("%.0f"), $3/$2 * 100}')
DISK_USAGE=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')

# =====
# RAPPORT
# =====

echo "==== SYSTEM REPORT =====" > "$REPORT_FILE"
echo "Date: $DATE" >> "$REPORT_FILE"
echo "CPU: $CPU_USAGE%" >> "$REPORT_FILE"
echo "RAM: $RAM_USAGE%" >> "$REPORT_FILE"
echo "DISK: $DISK_USAGE%" >> "$REPORT_FILE"
echo "SERVICE: ${SERVICES[@]}" >> "$REPORT_FILE"

log "Rapport créé: $REPORT_FILE"

# =====
# VERIFICATION SERVICE
# =====

for SERVICE in "${SERVICES[@]}"
do

    systemctl is-active --quiet "$SERVICE"

    if [ $? -ne 0 ]; then
        send_alert "Service $SERVICE arrêté"
        sudo systemctl restart "$SERVICE"
        log "Service $SERVICE redémarré"
    else

```



```

        log "Service $SERVICE OK"
    fi
done

# =====
# ALERTES SYSTEME
# =====

if [[ -n "$CPU_USAGE" ]] && (( $(echo "$CPU_USAGE > $CPU_MAX" | bc -l) )); then
    send_alert "CPU critique: $CPU_USAGE%"
fi

if [ "$RAM_USAGE" -gt "$RAM_MAX" ]; then
    send_alert "RAM critique: $RAM_USAGE%"
fi

if [ "$DISK_USAGE" -gt "$DISK_MAX" ]; then
    send_alert "DISK critique: $DISK_USAGE%"
fi

# =====
# TEST INTERNET
# =====

ping -c 1 8.8.8.8 > /dev/null 2>&1

if [ $? -ne 0 ]; then
    send_alert "Connexion internet perdue"
fi

# =====
# CLEANUP
# =====

find "$LOG_DIR" -type f -name "*.log" -mtime +30 -delete

log "Nettoyage terminé" >> "$LOG_FILE"

```

Il récupère automatiquement :

- l'utilisation du processeur (CPU) ;
- l'utilisation de la mémoire (RAM) ;
- l'occupation du disque.

Exemple de données collectées :

CPU : 15%

RAM : 42%

DISK : 37%

Ces informations sont enregistrées dans un rapport horodaté.

Capture d'écran 5 : Rapport généré

```
b04@b04-debian-server:~/Workspace/Reborn-Project/Automatisation/TP3/monitoring-devops$ ./monitor.sh
[10-06-2026 23:33:16] Rapport créé: /home/b04/Workspace/Reborn-Project/Automatisation/TP3/monitoring-devops/reports/report_10-06-2026_23-33-16.txt
[10-06-2026 23:33:16] Service nginx OK
[10-06-2026 23:33:16] Service ssh OK
[10-06-2026 23:33:16] Service ufw OK
[10-06-2026 23:33:16] Service fail2ban OK
```

```
b04@b04-debian-server:~/Workspace/Reborn-Project/Automatisation/TP3/monitoring-devops/reports$ ls
report_10-06-2026_23-33-16.txt
```

```
GNU nano 8.4
===== SYSTEM REPORT =====
Date: 10-06-2026_23-33-16
CPU: 4%
RAM: 11%
DISK: 11%
SERVICE: nginx ssh ufw fail2ban
```

7. Vérification des services

Le script vérifie également si les services Nginx, SSH, UFW et Fail2ban sont actifs.

Si le service en question est arrêté :

1. une alerte est générée ;
2. le service est automatiquement redémarré.

Commande utilisée :

```
systemctl is-active --quiet "$SERVICE"
```

Cette fonctionnalité constitue un mécanisme d'auto-réparation simple très utilisé dans les environnements DevOps.

8. Nettoyage automatique

Afin d'éviter l'accumulation de fichiers inutiles, le script supprime automatiquement les rapports plus anciens que 30 jours.

Commande utilisée :

```
find "$LOG_DIR" -type f -name "*log" -mtime +30 -delete
```

Cette opération permet de maintenir un espace disque propre et maîtrisé.

9. Automatisation avec Cron

Le monitoring est exécuté automatiquement grâce à Cron.

Configuration :

```
crontab -e
```

Puis ajout de la ligne :

```
*/5 * * * * /home/user/monitoring-devops/monitor.sh
```

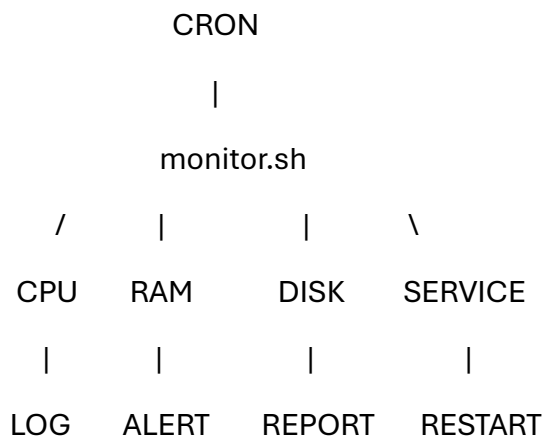
Cette planification exécute le script toutes les 5 minutes.

Capture d'écran 6 : Configuration Cron

```
GNU nano 8.4 /tmp/crontab.NezXB0/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
#
# Automatisation du script monitor.sh toutes les 5 min
*/5 * * * * /home/b04/Workspace/Reborn-Project/Automatisation/TP3/monitoring-devops/monitor.sh
```

10. Résultat global

Le fonctionnement général du système peut être représenté ainsi :



Le serveur est ainsi supervisé de manière autonome et peut réagir automatiquement à certaines pannes.

Conclusion

Ce projet a permis de mettre en œuvre plusieurs compétences essentielles en administration système et en DevOps :

- création de scripts Bash ;
- automatisation de tâches ;
- surveillance d'un serveur Linux ;
- gestion des journaux système ;
- déploiement automatisé ;
- exécution planifiée avec Cron.

Il constitue une première approche concrète des méthodes utilisées dans les environnements professionnels pour superviser et automatiser l'exploitation des serveurs Linux.